

OOT_用例_SSD_操作契约

OOT 开卷速查：用例、系统顺序图 SSD、操作契约

用途：期末开卷速查。

核心链条：用例 → 系统顺序图 SSD → 系统操作 → 操作契约 → 领域模型变化。

重点不是背定义，而是会把一个业务场景从“文字需求”转换成“系统事件”和“对象变化”。

0. 三者整体关系

用例 Use Case

↓ 从业务流程中找系统事件

系统顺序图 SSD

↓ SSD 中的消息就是系统操作

操作契约 Operation Contract

↓ 描述系统操作后的领域对象变化

领域模型 Domain Model

一句话区分：

内容	回答的问题	画/写什么
用例	谁为了什么业务目标和系统交互？	参与者、主成功场景、扩展场景
系统顺序图 SSD	外部参与者按什么顺序给系统发消息？	Actor → System 的系统事件
操作契约	某个系统操作执行后，领域对象怎么变？	前置条件、后置条件，重点是后置条件

最短口诀：

用例 = 业务故事

SSD = 黑盒消息顺序

操作契约 = 操作后的对象变化

1. 用例 Use Case

1.1 用例是什么

用例描述的是：

外部参与者和系统之间的一系列交互，这些交互最终为参与者提供业务价值。

用例不是系统内部类之间的调用，也不是领域类图。

1.2 三个基本概念

概念	含义	例子
Actor 参与者	与系统交互的外部对象，可以是人、外部软硬件系统、组织	业主、调度员、维修工、物业经理、第三方支付系统
Scenario 场景	参与者和系统之间的一次具体交互过程	业主成功提交一次报修
Use Case 用例	一组相关成功和失败场景的集合	提交报修、派工、提交评价、处理投诉

注意：

提交报修 = 用例

报修单 = 领域类

1.3 如何发现用例

按这个步骤：

1. 确定系统边界
2. 找外部参与者
3. 找每个参与者的业务目标
4. 用动宾短语命名用例

常见参与者

在物业报修系统中：

业主
调度员
维修工人
物业经理
外部支付系统
短信/邮件通知系统
库存系统

常见用例

提交报修
录入报修
派工
记录维修活动
提交评价
提交投诉
提交情况说明
处理投诉
查询报修状态
统计维修工工作量

不要把名词写成用例：

错误：报修单、评价单、调度记录
正确：提交报修、提交评价、派工

1.4 用例写法模板

用例名称：
主要参与者：
主要成功场景：

- 1.
 - 2.
 - 3.
- 扩展场景：
- 2a.
 - 3a.

考试一般写这个简化版就够。

1.5 示例：提交报修用例

用例名称：提交报修

主要参与者：业主

- 主要成功场景：
1. 业主向系统提交报修信息，包括故障描述、报修来源和联系方式。
 2. 系统记录报修信息。
 3. 系统创建报修单，并将状态设为“待调度”。
 4. 系统向业主返回报修编号或确认信息。

扩展场景：

- 2a. 如果故障描述为空，系统提示业主补充故障信息。
- 2b. 如果联系方式无效，系统提示业主重新输入。

1.6 用例常见考法

判断题

- 1. 用例描述系统内部对象之间的消息调用。错。
- 2. 用例描述外部参与者和系统之间的交互。对。
- 3. 参与者一定是人。错。
- 4. 用例可以包含成功场景和失败场景。对。
- 5. 用例是领域模型中的概念类。错。

简答题

给一段需求，找参与者和用例。

答题方法：

先找外部角色，再找每个角色想达成的业务目标。
用例名用动宾短语。

2. 系统顺序图 SSD

2.1 SSD 是什么

SSD = System Sequence Diagram，系统顺序图。

它把系统看成黑盒，只描述：

外部参与者向系统发出了哪些系统事件，以及这些事件的顺序。

SSD 不画系统内部对象。

2.2 系统事件与系统操作

概念	含义	例子
系统事件	外部输入、驱动系统的事件	调度员提交派工请求
系统操作	系统为响应系统事件而暴露的操作	assignRepairWorker(repairId, workerId)

通常在 SSD 中：

参与者 → System ： 系统操作(参数)

2.3 SSD 画图规则

SSD 只画：

Actor

System

Actor -> System 的消息

System -> Actor 的返回值，返回值可选

loop / alt 等简单控制结构

SSD 不画：

RepairService

RepairRepository

Database

RepairRequest

DispatchRecord

Controller

DAO

这些属于系统内部对象，不属于 SSD。

2.4 SSD 命名规则

系统事件应该用较高抽象层次的词语表达，不要用物理动作。

例如 POS 系统中：

较差：scan(itemId, quantity)

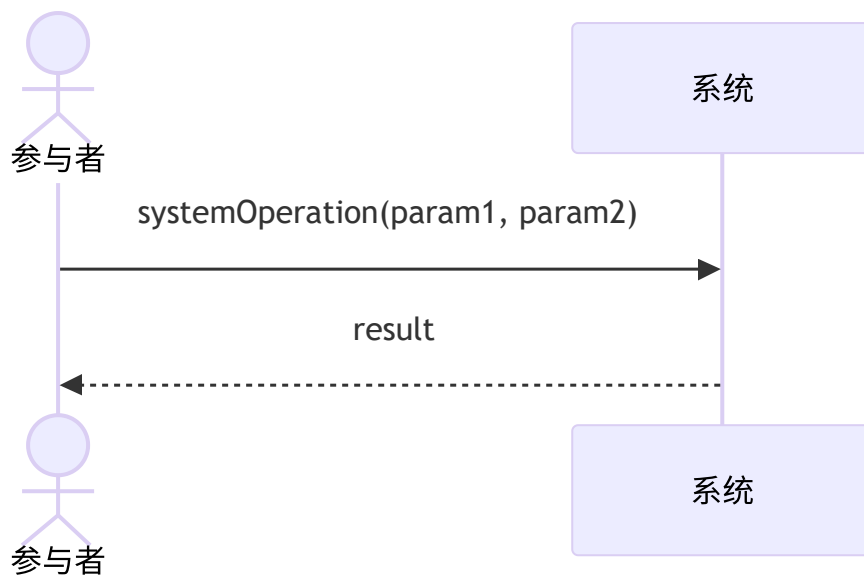
较好：enterItem(itemId, quantity)

物业系统中：

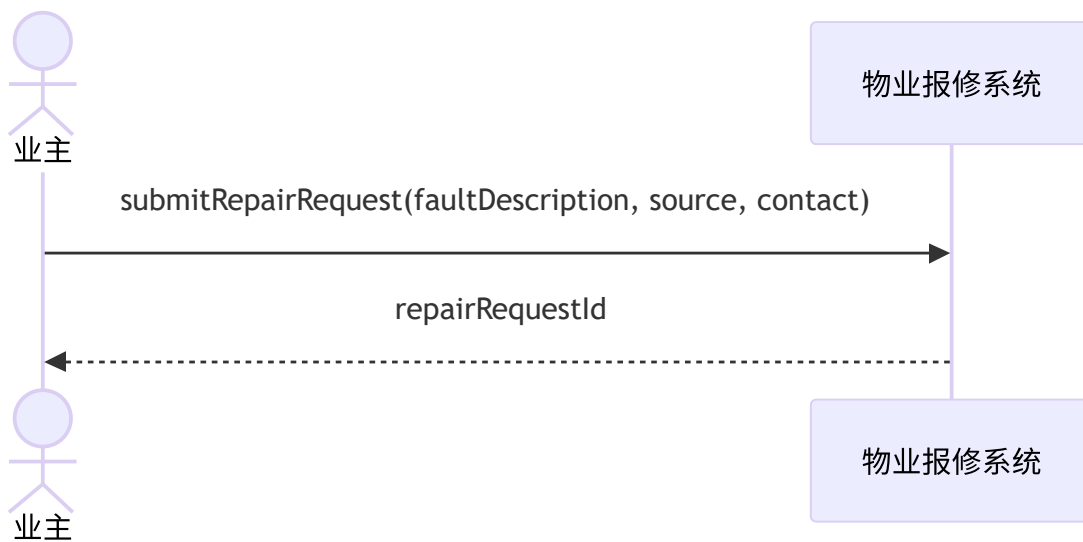
较差：clickSubmitButton()

较好：submitRepairRequest(...)

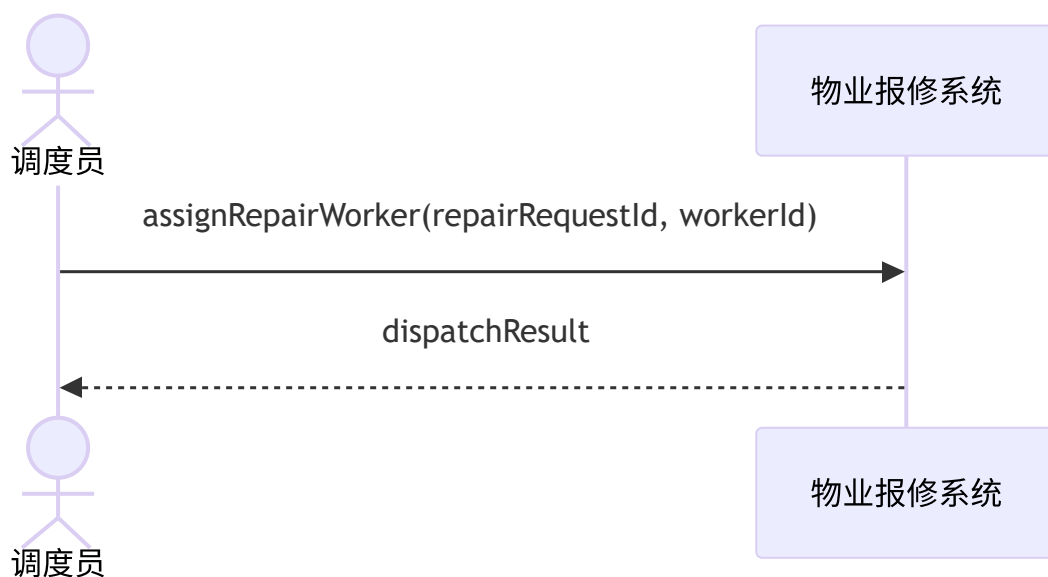
2.5 SSD Mermaid 模板



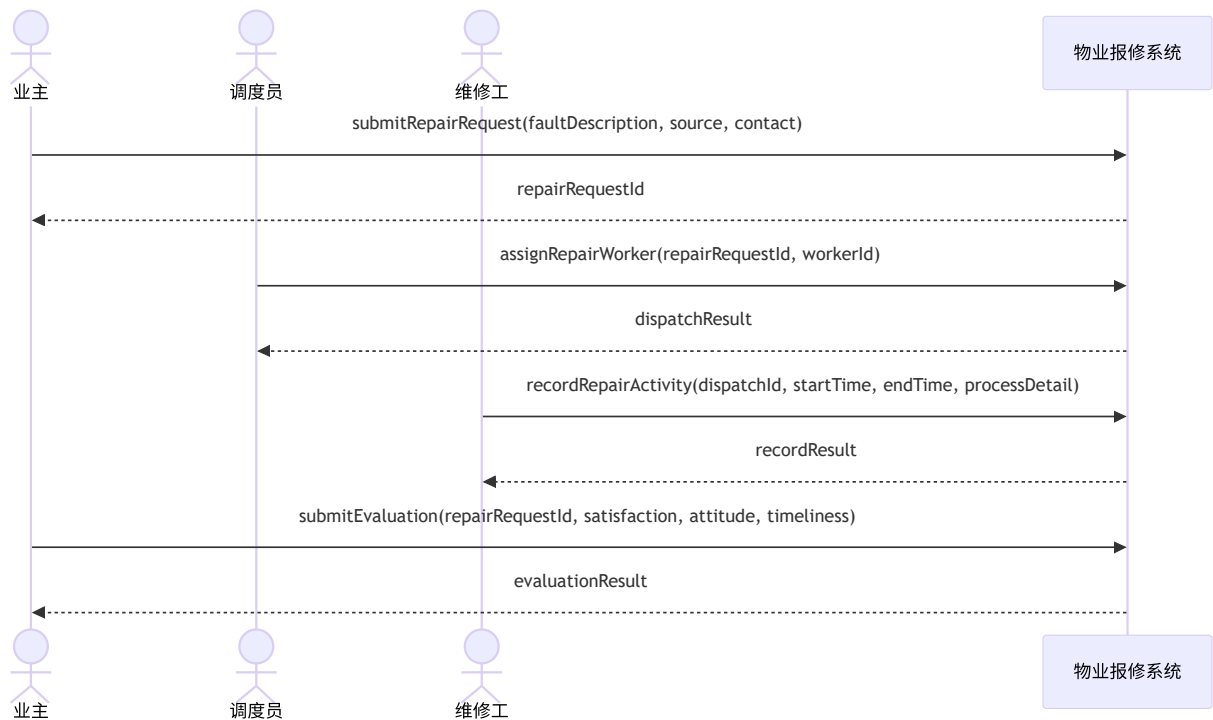
2.6 示例：提交报修 SSD



2.7 示例：派工 SSD



2.8 示例：报修处理完整场景 SSD



2.9 SSD 常见考法

判断题

1. SSD 把系统看作黑盒。对。
2. SSD 主要描述系统内部对象之间的调用。错。
3. SSD 中的消息通常对应系统事件或系统操作。对。
4. SSD 来自用例场景。对。
5. SSD 中应该画 Repository 和 Database。错。

画图题

给一段用例，让你画 SSD。

答题步骤：

1. 找参与者。
2. 把系统画成一个黑盒对象。
3. 从用例步骤里抽出外部输入。
4. 每个外部输入变成一条 Actor -> System 消息。
5. 必要时加返回值、loop 或 alt。

3. 操作契约 Operation Contract

3.1 操作契约是什么

操作契约用于进一步说明某个系统操作。

它重点描述：

系统操作执行后，领域模型中的对象、属性、关联发生了什么变化。

它不是代码，也不是内部方法调用。

3.2 操作契约什么时候需要写

一般不是每个系统操作都要写。

需要写的情况：

1. 系统操作比较复杂；
2. 操作结果在用例中不明显；
3. 操作会创建多个对象或建立复杂关联；
4. 需要精确说明领域模型状态变化。

简单查询类操作通常不必写复杂契约。

3.3 操作契约组成

操作：操作名称和参数

交叉引用：该操作来自哪个用例

前置条件：执行前系统或领域对象必须满足的状态

后置条件：执行后领域对象发生的变化

其中最重要的是：

后置条件

3.4 后置条件写什么

后置条件主要写三类：

1. 创建或删除对象实例
2. 修改对象属性
3. 建立或清除对象关联

不要写代码过程：

错误：调用 `RepairService.create()`

错误：调用 `repository.save()`

错误：执行 SQL `insert`

应该写领域变化：

正确：创建了一个 `RepairRequest` 实例。

正确：`RepairRequest.status` 被设置为“待调度”。

正确：`Owner` 与 `RepairRequest` 建立了关联。

3.5 后置条件写法风格

推荐使用过去时态，强调这是操作完成后的可观察结果。

较好：创建了一个 `DispatchRecord` 实例。

较差：创建 `DispatchRecord`。

3.6 操作契约模板

操作：

`operationName(param1, param2)`

交叉引用：

用例：`xxx`

前置条件：

- 1.
- 2.

后置条件：

1. 创建了……
 2. 设置了……
 3. 建立了……
 4. 修改了……
-

4. 物业系统常见操作契约模板

4.1 `submitRepairRequest` 提交报修

操作：

`submitRepairRequest(ownerId, faultDescription, source, contact)`

交叉引用：

用例：提交报修

前置条件：

1. 业主 `Owner` 已存在。

2. `faultDescription` 不为空。

后置条件：

1. 创建了一个新的 `RepairRequest` 实例。
2. `RepairRequest.requestTime` 被设置为当前时间。
3. `RepairRequest.faultDescription` 被设置为传入的 `faultDescription`。
4. `RepairRequest.source` 被设置为传入的 `source`。
5. `RepairRequest.status` 被设置为“待调度”。
6. `Owner` 与 `RepairRequest` 建立了“提交/发起”关联。

4.2 assignRepairWorker 派工

操作：

```
assignRepairWorker(repairRequestId, workerId)
```

交叉引用：

用例：派工

前置条件：

1. `RepairRequest` 已存在。
2. `Worker` 已存在。
3. 该报修单当前没有活动中的调度记录。
4. `Worker` 能处理该报修单对应的故障类别。

后置条件：

1. 创建了一个新的 `DispatchRecord` 实例。
2. `DispatchRecord.dispatchTime` 被设置为当前时间。
3. `DispatchRecord.status` 被设置为“活动中”。
4. `DispatchRecord` 与 `RepairRequest` 建立了关联。
5. `DispatchRecord` 与 `Worker` 建立了关联。
6. `RepairRequest.status` 被修改为“处理中”。

4.3 recordRepairActivity 记录维修活动

操作：

```
recordRepairActivity(dispatchId, startTime, endTime, processDetail)
```

交叉引用：

用例：记录维修活动

前置条件：

1. `DispatchRecord` 已存在。
2. `DispatchRecord.status` 为“活动中”。

3. `startTime` 早于 `endTime`。

后置条件：

1. 创建了一个新的 `ActivityRecord` 实例。
 2. `ActivityRecord.startTime` 被设置为传入的 `startTime`。
 3. `ActivityRecord.endTime` 被设置为传入的 `endTime`。
 4. `ActivityRecord.processDetail` 被设置为传入的 `processDetail`。
 5. `ActivityRecord` 与 `DispatchRecord` 建立了关联。
-

4.4 finishDispatch 完成调度

操作：

```
finishDispatch(dispatchId, finishTime, remark)
```

交叉引用：

用例：完成维修 / 完成调度

前置条件：

1. `DispatchRecord` 已存在。
2. `DispatchRecord.status` 为“活动中”。

后置条件：

1. `DispatchRecord.finishTime` 被设置为 `finishTime`。
 2. `DispatchRecord.remark` 被设置为 `remark`。
 3. `DispatchRecord.status` 被修改为“已完成”。
 4. 如果该报修单无需再次调度，`RepairRequest.status` 被修改为“待评价”或“已完成”。
-

4.5 submitEvaluation 提交评价

操作：

```
submitEvaluation(repairRequestId, ownerId, timeliness, attitude, satisfaction)
```

交叉引用：

用例：提交评价

前置条件：

1. `RepairRequest` 已存在。
2. `Owner` 是该报修单的提交者。
3. `RepairRequest` 已完成。
4. 该报修单尚未存在评价单。

后置条件：

1. 创建了一个新的 `Evaluation` 实例。

2. `Evaluation.evaluateTime` 被设置为当前时间。
 3. `Evaluation.responseTimeliness` 被设置为 `timeliness`。
 4. `Evaluation.serviceAttitude` 被设置为 `attitude`。
 5. `Evaluation.satisfactionResult` 被设置为 `satisfaction`。
 6. `Evaluation` 与 `RepairRequest` 建立了关联。
 7. `Evaluation` 与 `Owner` 建立了关联。
-

4.6 submitComplaint 提交投诉

操作：

```
submitComplaint(repairRequestId, ownerId, content)
```

交叉引用：

用例：提交投诉

前置条件：

1. `RepairRequest` 已存在。
2. `Owner` 是该报修单的提交者。
3. `content` 不为空。

后置条件：

1. 创建了一个新的 `Complaint` 实例。
 2. `Complaint.complaintTime` 被设置为当前时间。
 3. `Complaint.content` 被设置为 `content`。
 4. `Complaint.status` 被设置为“待处理”。
 5. `Complaint` 与 `RepairRequest` 建立了关联。
 6. `Complaint` 与 `Owner` 建立了关联。
-

4.7 submitStatement 提交情况说明

操作：

```
submitStatement(complaintId, employeeId, content)
```

交叉引用：

用例：提交情况说明

前置条件：

1. `Complaint` 已存在。
2. 员工 `Employee` 已参与该报修单的调度或维修处理。
3. `content` 不为空。

后置条件：

1. 创建了一个新的 `Statement` 实例。

2. `Statement.submitTime` 被设置为当前时间。
3. `Statement.content` 被设置为 `content`。
4. `Statement.status` 被设置为“已提交”。
5. `Statement` 与 `Complaint` 建立了关联。
6. `Statement` 与 `Employee` 建立了关联。

4.8 closeComplaint 关闭投诉

操作：

```
closeComplaint(complaintId, managerId, communicationRecord)
```

交叉引用：

用例：处理投诉

前置条件：

1. `Complaint` 已存在。
2. `Manager` 已存在。
3. 所有相关调度员和维修工均已提交情况说明。

后置条件：

1. `Complaint.status` 被修改为“已关闭”。
2. `Complaint.communicationRecord` 被设置为 `communicationRecord`。
3. `Complaint.closeTime` 被设置为当前时间。
4. `Complaint` 与 `Manager` 建立了“处理”关联。

5. 综合题答题流程

如果题目给出一段业务场景：

业主提交报修，调度员派工，维修工记录维修活动，业主提交评价。

可以按以下步骤答：

第一步：找用例

提交报修

派工

记录维修活动

提交评价

第二步：画 SSD



第三步：选一个系统操作写契约

例如 `assignRepairWorker(repairRequestId, workerId)`：

前置条件：

- 1. 报修单存在。
- 2. 维修工存在。
- 3. 报修单当前没有活动中的调度记录。

后置条件：

- 1. 创建了 `DispatchRecord`。
- 2. 设置了 `dispatchTime`。
- 3. 设置了 `status` 为“活动中”。
- 4. `DispatchRecord` 与 `RepairRequest` 建立了关联。
- 5. `DispatchRecord` 与 `Worker` 建立了关联。
- 6. `RepairRequest.status` 被修改为“处理中”。

第四步：反推领域模型

```
Owner 1 -- 0..* RepairRequest
RepairRequest 1 -- 0..* DispatchRecord
Worker 1 -- 0..* DispatchRecord
DispatchRecord 1 -- 0..* ActivityRecord
RepairRequest 1 -- 0..1 Evaluation
RepairRequest 1 -- 0..* Complaint
Complaint 1 -- 0..* Statement
```

6. 高频判断题

用例

1. 用例主要描述系统内部类之间的调用。错。
2. 用例是一组相关的成功和失败场景。对。
3. 参与者一定是系统用户。错。
4. 外部系统也可以是参与者。对。
5. 用例用于发现和记录需求。对。
6. 用例名称一般应根据参与者目标命名。对。

SSD

1. SSD 把系统看成黑盒。对。
2. SSD 中应该画出系统内部对象之间的调用。错。
3. SSD 展示某个用例场景中的系统事件顺序。对。
4. SSD 的消息通常来自外部参与者。对。
5. SSD 中的系统事件可以进一步对应系统操作。对。
6. SSD 里消息命名应尽量使用抽象业务词，而不是按钮点击、扫描等物理动作。对。

操作契约

1. 操作契约用于描述系统操作执行后的领域对象变化。对。
2. 操作契约主要描述 UI 页面跳转。错。
3. 操作契约的后置条件是最重要部分。对。
4. 后置条件应该描述对象创建、属性修改、关联建立或清除。对。
5. 后置条件应该写成具体代码调用过程。错。
6. 每一个简单查询操作都必须写完整操作契约。错。

7. 易错点总结

7.1 用例和领域类混淆

错误：报修单、评价单、维修工

正确：提交报修、提交评价、分派维修工

7.2 SSD 和设计顺序图混淆

SSD 只画：

调度员 -> 系统：assignRepairWorker(...)

不画：

系统 -> RepairService
RepairService -> RepairRepository
RepairRepository -> Database

7.3 操作契约和代码实现混淆

操作契约写领域状态变化：

创建了报修单。
设置了报修单状态。
建立了业主和报修单之间的关联。

不写代码过程：

调用 `createRepairRequest()`
调用 `save()`
执行 `insert SQL`

8. 考试速查模板

8.1 看到“找用例”

先找参与者，再找参与者的业务目标。
用例名写成动宾短语。

模板：

参与者：

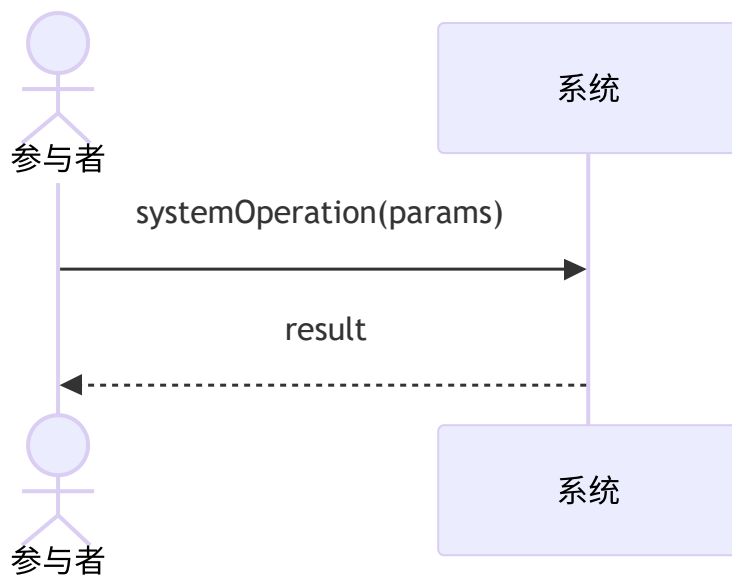
用例：

- 1.
- 2.
- 3.

8.2 看到“画 SSD”

只画参与者和系统。
消息 = 系统事件 / 系统操作。
系统是黑盒，不画内部对象。

模板：



8.3 看到“写操作契约”

从 SSD 中选一个系统操作。

写前置条件和后置条件。

后置条件写领域对象变化。

模板：

操作：

交叉引用：

前置条件：

后置条件：

1. 创建了……
2. 设置了……
3. 建立了……
4. 修改了……

9. 最终口诀

用例：谁为了什么目标使用系统。

SSD：参与者按什么顺序向系统发消息。

操作契约：这条系统消息执行后，领域对象怎么变。

用例不要写类名。

SSD 不要画内部对象。

契约不要写代码过程。